Target Specification, Not Model Capability, Explains Most Unit-Test Generation Failures at Medium Complexity

Nguyễn Tùng Ân, Đoàn Thị Thu Kim, Sơn Hải, Trương Hoàng Phúc, and Trần Xuân Lộc

Group 2 — SE1944, FPT University, Vietnam
`Supervisor: L.T.Q.Chi`

Abstract. Failures of LLM-generated unit tests are usually recorded against the model. Examining them individually, we find most belong to the benchmark and the measurements instead. Sampling by cyclomatic complexity alone admits targets no external test can reach: of 1,848 functions in the band $5 \leq CC \leq 10$ across ten projects, only 474 — 26 % — are simultaneously public and unambiguously named. We rebuild the benchmark under five testability criteria fixed in advance, sample 60 Java and 60 Python functions ($n = 59$ for Java after excluding one mis-sampled target), and score every suite at four nested tiers, resting all claims on mutation score inside the target's line range — the only tier a suite that verifies nothing cannot satisfy. Two one-shot prompts differing in exactly one variable isolate the effect of supplying the target specification. On Python it raises the proportion of suites reaching the target from 29/60 to 42/60 ($p = 0.003$, rank-biserial +0.61), but the proportion detecting a fault only from 16/60 to 22/60 ($p = 0.33$): the prompt improves arrival, not discrimination — a split branch coverage alone would have concealed. Against established generators the ordering reverses by language: the model beats Pynguin decisively on Python, where Pynguin detected faults on 2 of 60 targets under either configuration ($p \leq 0.003$ in every pairing), while on Java both EvoSuite and Randoop detect faults on more targets, significantly so only for EvoSuite under the stricter gate ($p = 0.028$). Six measurement defects surfaced, four introduced while repairing the other two; none produced an error message.

Keywords: Unit test generation · Large language models · Mutation testing · Cyclomatic complexity · Measurement validity

1 Introduction

A large language model asked to write a unit test for a particular function often fails to produce compilable code. It is tempting to blame the model, and we read it the same way in an earlier analysis of `gpt-4o-mini` on 120 medium-complexity functions. This paper reports on the investigation that led us to that conclusion

and explains why the answer is mostly about a benchmarking artifact rather than the model itself.

We found two issues, and together they compose a problem.

Benchmarks contain targets that no generator can test. Sampling functions by cyclomatic complexity cuts off the tail, but nothing else. If the function is not reachable from an external test, neither can the generator hope to test it, and asking it to do so will label it as failed. We found that in the ten projects studied, 1,848 functions fell in the desired complexity range of $5 \leq CC \leq 10$. Less than a third of them (474) were simultaneously public and had a clear name, suggesting that a generator that failed on the remaining 74 % would have been asked to do what no tool was capable of doing. Likewise, attribution of a method to the wrong declaring class renders a test suite invalid, and there is ample opportunity for that to happen: a brace counter that does not decrement on a method exit will assign the method to whichever nested type was last opened.

The usual metrics can be satisfied with no testing at all. Compilation success and coverage are both metrics that can be reported by a suite that never tests anything. Reflection turns every identifier into a string, so any call to a method that does not exist compiles cleanly with a zero return code; and a generated suite in our own corpus contained a single assertion-free call to the wrong function, which passed. Worse, the suite's metrics were all good on a superficial inspection. The issue was not one of noisy metrics; instead, the suite passed because it asserted nothing at all. We found six such cases, and in four of them, the defect was ours when repairing the other two (Section 6).

This paper

We build a new benchmark satisfying five testability criteria established in advance, sample 60 Java and 60 Python functions from the resulting set, and evaluate each test suite at four nested levels. We rely exclusively on mutation score within the line range as a metric. All other claims are vacuous for the suite that finds nothing because it detects no faults. Two variants of the prompt differ in the information given about the target, allowing us to report on the contribution of the target specification, while three established tools (EvoSuite, Randoop, Pynguin) act as reference points for comparisons. The details of sampling, tool versions, budget, and hypotheses were fixed in a public repository before measurements began.

Three findings stand out.

1. **Specification improves reach, not precision.** Adding a fully qualified target name, its signature, and a working constructor call to the prompt increases the proportion of suites reaching the target in Python from 29/60 to 42/60 ($p = 0.003$, rank-biserial +0.61). The number detecting a fault rises from 16/60 to 22/60, but this is not significant ($p = 0.33$). The prompt gets the test to the right place; it does not make it check anything. A study that

used coverage as its sole metric would have announced the change as an improvement.
2. **LLMs and search-based tools have their own merits.** The prompt conditions that add the target specification beat Pynguin across the board in Python. With Pynguin detecting a fault on only 2/60, compared to 16/60 and 22/60 for the two prompt conditions, they are better in every pairing at $p \leq 0.003$, and this applies to both of the configurations we tried for Pynguin. The trend is reversed in Java, where both search-based methods detect faults on more targets than the model — significantly so only for EvoSuite, and only under the stricter gate. These findings do not support one particular approach.
3. **Most of the attrition is structural.** The issue was never with writing assertions. It was with getting an instance of the subject under test, getting any assertions to pass, and generally navigating the maze of object creation that the search-based literature has already acknowledged for years. That is the single challenge that the model was unable to bypass.

We report the full sample size at all stages of the benchmarking funnel, and we discuss the two configurations of Pynguin at length. We describe an anomaly in the Java implementation of a green-test gate — the criterion requiring at least one test to pass against the unmodified source — that we found to contradict its own pre-registered specification. Where a repair procedure favors us, we report the effect size and the outcome in both configurations, so that nothing in this paper should depend on the reader's preference between the two.

2 Related Work

2.1 Search-Based and Random Test Generation

Automated unit-test generation has a long prehistory preceding the advent of language models. Randoop creates test sequences incrementally and uses feedback from executing each prefix to discard those that violate legality or redundancy [11]. EvoSuite treats an entire suite as an individual in a genetic algorithm and optimizes it using a coverage-based fitness function [3]. Both approaches have been intensively evaluated at scale: EvoSuite demonstrated its ability to achieve high branch coverage across a large sample of open-source classes, while also revealing that a significant proportion of classes remain challenging for reasons unrelated to the search itself [4]. Pynguin adapts the search-based approach to Python, where the lack of static typing decreases the information available to the test generator [8,9].

The two findings identified in this line of work are directly pertinent to our own. First, coverage and fault detection are not directly comparable: Shamshiri et al. find that automatically generated suites with high coverage detect only a minority of real faults [14], and Inozemtseva and Holmes find that coverage's correlation with suite effectiveness is significantly lower than often assumed [5]. Secondly, the challenge of the target is not necessarily captured by its complexity:

the construction of the receiver object often proves to be the major stumbling block for test generation. We will return to both observations in Section 3.3 and Section 5 respectively.

2.2 LLM-Based Test Generation

Code-trained language models transformed test generation into a prompting task [2]. The earliest approaches in this space involved fine-tuning a transformer on method–test pairs with surrounding focal context [16]. Later works switched to prompting. TestPilot produces JavaScript tests from documentation and usage examples and employs a requery mechanism triggered by test failures [13]. ChatTester introduces an iterative repair loop that capitalizes on compiler and runtime errors, yielding markedly better results than a single-shot approach [18]. Siddiq et al. evaluate several LLMs' ability to generate JUnit tests and find compilation failures to dominate over weak assertions as a source of failures [15]. Finally, a recent survey enumerates the space and reflects on the evaluation practices, noting the disparity between different approaches [17].

Our study is narrower than most of these works in one domain (we only utilize a single prompt per function) and stricter in another (we do not leverage iterative repair). This choice removes the repair loop that these systems often rely on, because a repair loop obscures the initial prompt's ability to synthesize correct code on its own, which is our main concern. Furthermore, Section 3.2 varies the prompt in only one condition to probe whether this difference is the cause of the discrepancy. By contrast, these approaches attempt to maximize the success rate given a fixed prompt.

2.3 Evaluation Metrics and Their Validity

Mutation analysis is the canonical means of assessing test-suite adequacy in fault-based evaluation. Mutants have been shown to be a reasonable if imperfect substitute for real faults in controlled experiments [6,12]. It is also computationally demanding, which is why many works on LLM-based test generation focus on compilation success and coverage as surrogates for adequacy.

This substitution forms the theoretical basis for our own contribution. Compilation success and coverage are easy to measure, but they also provide less information than one might hope: a reflective call compiles regardless of the target method's presence, and a suite that constructs no object and calls nothing still passes. Neither case actually evaluates the code under test, but both outcomes are entirely possible under the metrics used in existing works. We have observed both scenarios in our own corpus (Section 3.3), which is why we report four nested tiers. Any claim only pertains to the most stringent level that is not trivially achieved by a prompt that merely fulfills the measurement's requirements.

Another subtlety involves the validity of the benchmark under consideration. Selecting targets based on cyclomatic complexity [10] is common practice, but complexity is not necessarily indicative of reachability. A method can be private,

have multiple overloads share a name, or belong to a class not instantiable from outside the target package. A suite that fails to address such a method tells us about the sampling, not about the generation approach. To our knowledge, the proportion of complexity-sampled methods addressable from external tests is rarely quantified: we report the funnel effect, which appears to be of general interest.

2.4 Methodological Practice

Because the current work undertakes a replication investigation that re-analyzes a dataset contaminated by discoveries made on another, the distinction between remedying an instrument and engaging in wishful thinking is particularly crucial. Kerr's articulation of the hazards of harking [7] is especially pertinent to our endeavor, which seeks to avoid data-dredging by employing a priori decisions. We have committed our sampling criteria, tools, budgets, and hypotheses to the repository before commencing measurements. In addition, the dataset used in the previous study is not overwritten; instead, this paper provides an alternative analysis alongside the original findings. For the present work's statistical evaluations, we adhere to the recommendations of Arcuri and Briand regarding the selection of non-parametric tests and the consideration of effect sizes in randomized software engineering experiments [1].

3 Methods

3.1 Dataset Construction

We extracted functions from ten open source repos at a particular commit (eight java projects: `commons-cli`, `commons-csv`, `commons-collections`, `commons-math`, `gson`, `jsoup`, `joda-time`, `jfreechart`; two python: `flask`, `requests`) using Lizard 1.23.0 to compute cyclomatic complexity (CC). We select the medium complexity band $5 \leq CC \leq 10$ as in our previous work.

While complexity is a precondition for being a challenging test target, a study sampling on complexity confounds two phenomena: the quality of tests produced by the generator, and the possibility that the sampled target is actually reachable by the generator. We therefore applied five additional criteria, each of which was considered necessary for an LLM, search-based generator, or human to reach the target via the public API.

F1 — **Non-trivial body:** $nloc \geq 3$. This filters out stubs, abstract methods, and interface signatures that specify no branches.

F2 — **Public or exported:** Java: declared `public`. Python: not prefixed with an underscore. A `private` method cannot be invoked directly by an external test class, unless the test uses reflection to bypass access controls. However, the compiler no longer checks any qualification appearing in a dynamic method call (Section 3.3), so compilation ceases to be evidence of anything.

Table 1. Sampling funnel. Each row reports how many candidates survive the criterion. F4 corrects the declaring-class attribution that F3 depends on rather than removing candidates in its own right, so it has no separate row.

Stage	Criterion	Remaining
F0	$5 \leq CC \leq 10$ (Lizard)	1,848
F1	non-trivial body	1,848
F3	unambiguous name	753
F2	public / exported	474
F5	dynamically resolvable (Python)	64
	final pool	Java 406, Python 64
	sampled	Java 60, Python 60

F3 — Unambiguous name: Java: the method name occurs exactly once in the declaring class; Python: the function name occurs exactly once in the defining module. An overloaded name cannot appear in a prompt without additional disambiguating context, and the compiler will reject the invocation as ambiguous.

F4 — Verified declaring class: The enclosing class is found with a brace-depth stack that is popped on exit. Taking "the nearest enclosing class declaration" to mean the one most recently opened, one could incorrectly attribute a method to whichever nested class was most recently declared in the file.

F5 — Dynamically resolvable: (Python) The module is imported, the qualified name is walked with `getattr`, and the result is checked for being callable. Syntactic resolution is not sufficient evidence for run-time binding.

Table 1 shows the complete set of criteria. We report all stages of the filtering pipeline, as the attrition is informative: of the 1848 functions that passed the complexity filter, only 474 (26 %) were both public and unambiguously named. Around 3/4 of medium complexity functions in these projects were not valid test targets for an external generator.

From the final pool we sample 60 Java and 60 Python functions, stratified across repositories (3–9 functions per java repo; 32 from `requests`, 28 from `flask`). Complexity is skewed towards the lower end of the band (CC=5: 56), with the median function containing 17 lines of code. The Python targets comprise 34 instance methods and 26 module functions. Every one of the 120 functions was checked for run-time accessibility before generating tests. One Java target was later found to violate F2 — our own parser had recorded a `protected` method as public — and is excluded, so Java results are reported over $n = 59$ and Python over $n = 60$ (Section 6).

3.2 Test Generation

All tests are generated by `gpt-4o-mini-2024-07-18` at `temperature`=0, `top_p`=1.0, `max_tokens`=2048 in a single invocation per function with no feedback loops or

retrieval. We compare two one-shot prompt conditions. Both contain exactly one worked example (a trivial `add` function and test), so both are one-shot in the usual sense, and both make one API call. The conditions differ in one respect:

V1 — baseline. The prompt contains the target function name and source.
V2 — target-specified. Same as V1, but with one block inserted before the task description: the fully qualified target name, its signature, and declaring class, plus — for instance methods — an executed constructor call that has been verified to produce a valid receiver object.

The necessity of a worked example is crucial: an earlier variant of the enriched prompt accidentally omitted the example and thus differed from V1 in two respects (no example plus metadata). We report this variant only as a secondary observation and label it zero-shot, since it is effectively one.

3.3 Measurement: Four Tiers

A simple binary metric fails to distinguish several important degrees of success, so we report four nested tiers. Each subsequent tier implies the one before, and importantly, the first two can be achieved by a test suite that does not touch the target at all.

T1 — Compiles / collects. The test file parses and collects no errors.
T2 — At least one green test. At least one test passes when run against the unmodified source.
T3 — Reaches the target. Achieves branch coverage $> 0\%$ inside the target's line range [start, end].
T4 — Detects faults. Achieves mutation score $> 0\%$ in the same range.

Only T4 speaks to the ability of a suite to test the target; we demonstrate that T1 and T2 can be achieved even when a suite provides no tests at all:

- **T1 under reflection.** Name qualification disappears under dynamic method invocation: the name becomes a string the compiler stops checking. A call to `getDeclaredMethod("aNameThatDoesNotExist")` compiles with return code 0 and throws only at run time. Suites that employ reflection will therefore have artificially inflated validity rates.
- **T2 under an assertion-free suite.** The search-based suite generated for `requests.hooks` (target `dispatch_hook`) contains only one test, which in turn is a bare call to another function in the same module, with no assertions whatsoever. This suite "compiles", "collects", and "passes" — T1 and T2 are both satisfied — while coverage within the target's line range is 0% (T3 not achieved). A green-test criterion records this as a positive result.

The two tiers are therefore not merely weaker measures than T4; they can be achieved by suites that are demonstrably ineffective at testing the target.

Branch coverage is measured with `coverage.py` (python), attributed to the line range of the target rather than the whole file; T3 is therefore reported for Python only. The Java pipeline runs the generated suites through the JUnit platform launcher and rests on T4, so no Java branch-coverage figure is claimed. Mutation analysis uses the same operator set for both languages: arithmetic ($+/-$, $\times/\div$), relational ($>$ / $\geq$, $<$ / $\leq$), equality ($==$ / $\neq$), boolean (AND/OR), boolean constant inversion, and numeric constant increment, capped per function by language (20 mutants for every Java suite, 30 for every Python suite) and limited to the target's line range. The cap binds two Java functions and no Python one (Section 6); it does not affect T4.

Two procedural notes apply to all measurements and should be considered part of the methodology. First, mutation testing requires rewriting the source under test, which, run on a shared tree, would silently corrupt any concurrent measurement. We mitigate this by injecting the mutated files into the import path ahead of the original tree, so that concurrent measurements interfere with each other as little as possible. Second, the versions of the tools must be cross-compatible (JUnit 5 pairs Jupiter 5.x with Platform 1.x), or the launcher will fail to find any tests at all and report success anyway when run with a Jupiter 6 artifact.

3.4 Baselines

Each baseline is run per class (java) or per module (python), as appropriate to the tool, and results are attributed to individual functions by line range, as for the LLM suites.

EvoSuite 1.2.0: 60 s per class, JDK 11. EvoSuite uses deep reflection to probe the JDK, which is disallowed by the module system in Java 9+. Its master process launches the client from `JAVA_HOME`; therefore, the two locations must agree, or the client will fail to start while the master reports success. As the projects were built with JDK 17, we rebuilt the source code as Java 11 bytecode in a separate directory.

Randoop 4.3.3: 60 s per class, JDK 17.

Pynguin 0.45.0: 90 s per module, two variants reported. By default, Pynguin serializes module state to a worker process with `dill` and fails on C-implementation types that resist name resolution, resulting in no output for several of the tested modules. We therefore report Pynguin's default behavior in one set and the same tool with the master–worker split disabled in the other. This disentangles "the tool scored poorly" from "the tool did not run".

3.5 Statistical Analysis

We compare all conditions with the paired Wilcoxon signed-rank test, matched on the function identifier, and report all results at the $\alpha = 0.05$ level, with the matched-pairs rank-biserial correlation as the effect size. Java and Python are

Table 2. Fault detection rate (T4), original versus rebuilt set, same prompt (V1).

Language	Green gate	Original	Rebuilt
Python	per-test	13/60 (21.7 %)	16/60 (26.7 %)
Java	whole-suite	6/60 (10.0 %)	12/59 (20.3 %)

analyzed separately throughout: the toolchains are different, and Python has no access control while Java does, so the two languages have different failure modes.

Within each language, rates are reported over the functions of that language (59 for Java after the exclusion above, 60 for Python), not per repository.

Results are reported at two levels: all ($n = 59$ for Java, $n = 60$ for Python, with zeroes counted) and effective (the subset with a non-zero value). The all level is reported even when the effective level is better.

All the choices about the prompt format, criteria, tool versions, timing budgets, the Pynguin two-branch design, and the hypotheses to evaluate are committed in a pre-registration before any baseline is run.

4 Results

We report results per language across all metrics. Within a language, all rates are percentages over the same n: 60 for Python, 59 for Java (after dropping the mis-sampled target in Section 6). The top level, T4 (fault detection within the target's line range), dominates the next, T3 (reaching the target), which is why we report both in a joined table.

4.1 RQ-A: Clean Versus Original Dataset

Table 2 compares the baseline prompt's fault detection rate on the original set to the same prompt on the rebuilt set: not a fair comparison, since the sets contain different target functions, and the rebuilt is easier by construction. We report only the magnitude of the difference and not the statistical significance, since measuring the latter would require the same green-test gate (to avoid differences in the set of tested targets inflating p-values beyond any real effect).

The rebuilt set increases Java's rate by about 100%, and Python's by 25% (13/60 to 16/60). As a reminder, these are not comparable sets: the two are different sets of functions, and the rebuilt one is easier by construction. We have no particular hypothesis about the effect of rebuilding beyond a warning that much of the original set's poor performance derived from targets that could not reasonably be covered by tests.

4.2 RQ-B: Effect of Target Specification

Both prompt variations are one-shot; the difference is in one block of target metadata (Section 3.2). Table 3 reports the same tiers for Python, where the

Table 3. Python, V1 versus V2. Paired Wilcoxon signed-rank, $\alpha = 0.05$; r_{rb} is matched-pairs rank-biserial.

Tier	V1	V2	p	r_{rb}	
T3 (reaches target)	29/60	42/60	0.0027	+0.61	significant
T4 (detects fault)	16/60	22/60	0.3259	+0.28	n.s.

effect is strongest and most cleanly separated between the tiers. We report the median of the non-zero subsets; the non-zero Python results are all much higher than the zero group, so this avoids suppressing interesting variation by including the 0% scores.

The target specification increases the proportion of suites that reach the target (29/60 to 42/60) and has a non-significant effect on the proportion of suites detecting the injected fault (16/60 to 22/60). On the non-zero subsets, the medians of both T3 and T4 (75.0, 84.4) are barely changed (79.2, 88.9), which suggests the effect is almost entirely in the number of suites passing each tier rather than their ability to perform once they do. A coverage-only metric would report an improvement for T3; for T4 it is not enough for the test to pass, but it does pass more often.

The pre-registered hypothesis H-B predicted that the enriched prompt would not outperform the baseline condition. It holds for T4 (detecting the injected fault) and is contradicted for T3 (reaching the target). We report this as a partially refuted hypothesis, since the significant T3 effect is in the direction H-B predicted against.

On Java, the same intervention has little effect. With the per-test gate, both prompts detect the fault on 16/59 targets; with the whole-suite gate, 12 (V1) and 10 (V2) respectively. The number of compile-failure targets is the same, 38, under both prompts (35 of the 38 are the same functions). The target block, which identifies the target uniquely, does not resolve the type-level issues that dominate Java compilation failures. There are only 4 functions where the prompt variations differ in a measurable way, below the 6-pair minimum needed for a Wilcoxon test; we report the observed counts and the mechanism (Section 5.1) rather than a p-value.

4.3 RQ-C: Against Established Generators

Table 4 pits the model against search-based and random test generators. Java is reported under both green-test gates; Python under both Pynguin settings. No analysis below depends on adopting one versus the other.

The languages point in opposite directions: on Python, the model overwhelmingly outperforms Pynguin under all settings, but on Java the baselines outperform the model in most settings. The reason for the difference is visible in the counts: on Python, Pynguin fails to generate a test suite for 39/60 targets in its default configuration, and 35 in the no-split configuration; disabling the split (the stated fix for the serialisation crash) leaves the fault detection rate unchanged,

Table 4. Fault detection (T4) of the model versus baselines. Java $n = 59$, Python $n = 60$. GPT columns are the target-specified prompt (V2); the paired test compares that column with each baseline.

Language Setting	GPT (V2)	Baseline Paired test (p, r_{rb})
Java — EvoSuite		
per-test	16/59	23/59 0.14, +0.32 (n.s.)
whole-suite	10/59	24/59 0.028, +0.51 (sig.)
Java — Randoop		
per-test	16/59	19/59 0.86, −0.04 (n.s.)
whole-suite	10/59	18/59 0.49, +0.18 (n.s.)
Python — Pynguin		
default	22/60	2/60 0.0001, −0.96 (sig.)
no split	22/60	2/60 0.0001, −0.96 (sig.)

because the net movement from 39 to 35 conceals 17 targets gaining a suite and 13 losing one, with 14 of the 17 new suites failing at collection. On Java, the baselines detect the fault on more targets than the model in all comparisons (EvoSuite 23 or 24 to the model's 16 or 10; Randoop 19 or 18 to 16 or 10).

The comparisons are not uniformly significant. On Python, the paired Wilcoxon test rejects the null against Pynguin with a p-value of 0.0001 and rank-biserial correlation of −0.96 under both settings. On Java, only the EvoSuite comparison under the whole-suite gate is significant ($p = 0.028$, rank-biserial +0.51), while the other three Java comparisons (EvoSuite per-test, and both Randoop comparisons) all had p-values ≥ 0.14 and were not significant at $\alpha = 0.05$. This means the Java evidence is a consistent count advantage for the search-based tools that is statistically firm only for EvoSuite once the stricter whole-suite gate is applied; it is not the decisive reversal the raw counts might suggest. Even so, it points the opposite way from Python, where the model rejected the Pynguin baselines at $p \leq 0.003$ in all four settings. Java is the language for which these tools were designed, and where static types give a search-based generator the call information the model was shown to lack (Section 5.1). We therefore report no overall conclusion from these comparisons: the count advantage was reversed in the two languages, and a study limited to one would have falsely concluded the opposite for the other.

5 Discussion

5.1 Three Barriers, Only the First of Which a Prompt Can Remove

The two prompt conditions differ by only one variable, and the effect of that variable cleanly splits between languages. For Python, adding the target spec cut the number of suites that failed to import or collect from 17 to 4 — 14 saved targets and 1 additional failure. For Java, it made virtually no difference: 38 failed to compile with the baseline prompt, and 38 with the target-specified

one, with 35 functions overlapping between those sets. Three functions moved in either direction, which is not enough for a meaningful result.

The difference in effect between languages is not a peculiarity of their tools — it is encoded in the block itself. The block specifies the identity of the target: its qualified name, declaring class or module, signature, and a constructor call for the receiver, if any. For Python, that is enough to remove barriers to invocation — at the stage these suites failed, the failure was always importing a module from an incorrect path, calling a non-existent function, or attempting to invoke a method on a class that does not support it. Give Python the correct name and location, and it will resolve the symbol successfully. Java performs these checks at compile time, so a correct name is still insufficient — the model must synthesise type-correct uses of the symbol within the suite, including receiver arguments of the correct arity and parameter types, return types, thrown exceptions, generics, and so on. The target block does not include any of that information.

This suggests a three-level reading of the results in terms of what the generated test must do to observe the target. We offer this as a hypothesis for future work:

1. **Addressing** — calling the right thing at all. The target specification removed this barrier, almost entirely in Python.
2. **Typing** — calling it in a way the compiler accepts. Only relevant to Java. The target spec does not meaningfully help, because the difficulty is not with the target itself, but the surrounding API.
3. **State** — constructing a receiver in a configuration where the behaviour of interest is visible. Neither condition addresses this.

The reason to take this seriously is the evidence in support of the third item. For Python, the number of suites that imported successfully but had no passing tests changed little between the baseline prompt (13/60) and the target-specified one (14/60), with only 7 functions appearing in both sets. The problem was not with addressing — the same workload had to find tests for roughly the same set of functions. Removing the addressing barrier did not shrink the state barrier; it exposed it. The population changed while its size did not.

The observation that the barrier is not with assertion writing but with receiver construction is not new — it is the reason search-based test generation struggles with object-oriented code [4,14]. The point of this finding is that the same wall exists for language models, and that providing the constructor is not enough to get past it — our snippets were run and checked to create the receiver, and the state barrier persisted. Building an instance is not the same as building an instance in a state where some behaviour is triggered.

5.2 Reaching a Target and Testing It Are Different Achievements

The clearest finding of this study is the difference between two tiers that would have appeared identical in a single-metric study. On Python, the target spec increased the proportion of suites that reached the target from 29/60 to 42/60

($p = 0.003$, $r_{rb} = +0.61$) but had no significant effect on the number that could detect an injected fault, from 16/60 to 22/60 ($p = 0.33$).

A study that used only coverage as a metric would have concluded that the enriched prompt is an improvement across the board, while also being wrong about what aspect of testing it improved. The suites arrived at the right function more often, but tested it no more often. Coverage measures arrival and mutation score measures discrimination — only the latter is a metric of testing ability in any meaningful sense, and only the former was improved by the prompt change. That coverage is misleading about test suite quality is a well-known finding [5]; what this experiment adds is a demonstration that a specific, plausible change to the prompt moves the two metrics by different amounts, in a way that inverts the conclusion drawn from them.

This is also where the pre-registered hypothesis H-B begins to fall apart. We predicted, with no knowledge of the results, that the enriched prompt would be no better than the baseline. That prediction holds at T4 and fails at T3. We report this as a partially refuted hypothesis, rather than an overturned one, because the direction of effect at T3 is the one we predicted against.

5.3 Neither Family Wins Outright

The comparison with existing generators also fails to reveal a clear winner, and the languages rank oppositely.

On Python, both prompt conditions substantially outperformed Pynguin: 16/60 and 22/60 suites passing compared to 2/60, $p \leq 0.003$ in all pairs, with rank-biserial correlations ranging from -0.81 to -0.96. The reason for the gap was apparent in the numbers of failed suites: Pynguin failed to generate any suite for 39/60 targets in its default configuration. The documented workaround for the reported serialisation crash (disabling the master–worker split) reduced this to 35 targets, a net movement of four that conceals 17 targets gaining a suite and 13 losing one, with 14 of the 17 new suites failing at collection, leaving T4 at 2/60. We report this result because it is the sort of finding that disappears when a study only reports the better-performing configuration. In this case, neither had any.

On Java, the ordering was reversed: the search-based tools found faults on more targets than the model in all pairs (Section 4.3). It was stronger for EvoSuite, for which the paired test reached significance at the whole-suite gate ($p = 0.028$), than for Randoop, for which none did; we interpret this as a consistent advantage for the mature tools, rather than an outright win for any. Java is also where these tools were developed and where they have been iterated upon for the last decade, and where static typing gives them the information needed to construct tests — the information the language model was demonstrated to be unable to reproduce in Section 5.1. Removing it from consideration explains the reversal.

The conclusion is that neither family of tools is demonstrably better overall, but that they fail in ways that are specific to languages. A study that examined

only Java would confidently report the superiority of search-based testing; a study that examined only Python would report the opposite.

5.4 What This Means for Evaluating Test Generators

Three practices follow from what went wrong rather than from what worked.

Report the sampling funnel. Only 26 % of medium-complexity functions in these ten projects are both public and unambiguously named. A benchmark that does not consider name ambiguity will spend most of its effort on targets that no external tool can possibly invoke, and every tool evaluated on it is charged for failures that belong to the sampling. This consideration should shape any benchmark that aims to evaluate test generators; reporting the shape of the funnel is nearly mandatory if one wishes to honestly report a tool's ability to test real code.

Rest conclusions on a tier that cannot be satisfied vacuously. Compilation success is defeated by reflection, which turns names into unchecked strings. The green-test criterion is defeated separately: a suite that makes one assertion-free call to the wrong function satisfies it — it is recorded as a success — yet tests nothing, and this is a common occurrence in this corpus. Neither observation requires an adversary to appear; both were observed in standard tool output. Metrics should be interpreted with extra caution in the presence of reflection, or similar techniques that allow code to bypass static checks.

Assume the harness is broken until a failure proves otherwise. The failure mode that consumed the most time in this study was not a tool underperforming, but rather a metric misreporting. Hardcoded constants, validity flags erroneously indicating success, reflection turning method names into strings the compiler never checks, version mismatches reporting no tests while a suite exits successfully, JDK mismatches doing the same, and a green-test gate implemented differently in the two languages under one name appeared six times in total, four of which were introduced by us while attempting to fix the other two. None of these issues caused an exception to be thrown, and consequently, none of them were caught by the initial harness design. The requirement for a harness is that it must have a test that fails — if the pipeline below the metric is entirely broken but reports no change, that metric is no longer useful. We enforce this requirement for all reported rates, and it is the cheapest such check we know of.

6 Threats to Validity

6.1 Construct Validity

What a validity flag measures. The main construct validity concern is that our suite's "valid suite" marker does not capture what it ought to; specifically, that we fail to detect when it does not. There were six cases, four of which were in our instrumentation:

1. A compilation flag inferred from a coverage report only captures whether the class appears in it; coverage tools list all classes (including unreachable ones), which yield zero percent branch coverage (parsed as a successfully compiled function with zero coverage).
2. A criterion of "at least one test executed" erroneously counts a file in which all tests failed.
3. Reflection renders the compiler-based name checking useless for any suite making use of it.
4. A discrepancy in tool versions may cause a test launcher to report zero tests found while exiting successfully.

The fifth case has ramifications for the statistical test, so we describe it in detail: our Java runner and the tests it generated were compiled with the default JDK (class file version 61) and run on JDK 11 (required by EvoSuite; reads class files up to version 55), causing an `UnsupportedClassVersionError` to be thrown by the JVM. Because the exception was thrown inside a child process, the harness saw no result line and reported "no tests ran" (for all targets). EvoSuite scored 0/60, and the paired test against it had $p = 0.005$ and rank-biserial correlation of -1.000; had we reported it, the paper would have erroneously claimed a significant advantage for the model over EvoSuite. The correct value is given in Section 4: a well-formed p-value made the artefact appear stronger than it was.

Finally, our green-test gate instrumentation correctly kept passing tests and discarded failing ones in the Python tool, as T2 specifies; the Java version rejected the whole function if any test had failed. The two languages implemented different constructs under the same label, and the Java version contradicted our pre-registered definition. We fixed it and report results with both settings henceforth (Section 4); the correction is relevant to the conclusion validity (see below), as it improved our results.

None of these issues produce visibly incorrect results; each affects specific metrics, however, and we report the four nested levels of Section 3.3 instead. We consider only T4 — detection of faults within the target's line range — as a validity flag for any suite.

Mutation operators. Our operator set was deliberately limited and identical for both languages for the purpose of comparison, and it is weaker than a production mutation framework; therefore, absolute scores on the same code base are not directly comparable to the results of PIT or `mutmut`. We also note that the per-function mutant cap differs by language — 20 for Java, 30 for Python — and binds only two Java functions (CJ-004 and CJ-026) and no Python one, so it affects the resolution of the score rather than the tier outcome.

6.2 Internal Validity

Sampling. Our own miner incorrectly attributed one Java target's visibility: `LocalTime.getField` was declared `protected`, but the parser matched it against

a nested class's public method with the same name and therefore passed the public visibility check. An audit against the real declaration line confirmed that this was the only occurrence among the 60 Java functions; we excluded it from analysis. We report this case, however, because it demonstrates a class-attribution error similar to the one this study seeks to detect, albeit in our own tooling: knowing about a failure mode does not protect us from it.

Concurrency. Mutation analysis overwrites source files; therefore, running it on a shared source tree corrupts any subsequent measurement reading from it, silently. We mutate an isolated copy placed before the original on the import path and verify that the original tree was not modified after each run.

6.3 External Validity

Python's source exhaustion. After applying all testability criteria, there were only 64 Python functions in the two projects under analysis, 34 of which had already been present in our previous dataset. Therefore, a strictly held-out Python replication study can only use approximately 30 Python functions, without additional repositories. The Python corpus was also drawn from two web-oriented libraries, potentially limiting external validity.

Code complexity. While the complexity range of $5 \leq CC \leq 10$ is broad, most of the code in the sample falls towards the lower end of it (56 of 120 functions): there were more functions with complexity 5 than 7–10 combined. Therefore, conclusions about the upper half of the range may be harder to draw.

Model and config. We evaluated only one model at one temperature; we made no claims about its generalisability, and the deterministic setting does not let us evaluate the variance introduced by different runs.

6.4 Conclusion Validity

Baseline config. Three factors affect the baseline comparison adversely: first, EvoSuite was run with a classpath only including the module's own compiled classes; with the full classpath, its bytecode parser fails spectacularly (without any visible output) and we were unable to isolate the offending dependency. Therefore, EvoSuite may have been performing worse than reported in a real-world scenario. Second, seven of the Java targets in the multi-module project could not be recompiled with Java 11 bytecode (needed by EvoSuite) and were excluded from consideration. Finally, our Randoop version differs from the one used in our previous study; therefore, its results are only directly comparable to those of the same toolchain.

Pre-registration and two deviations. The research questions, hypotheses, tool versions, budgets, and the two-branch Pynguin design were committed to the pre-registration before any baseline was run.

The first deviation added a condition; upon inspection, the enriched prompt we intended to use did not include the worked example, making it a zero-shot prompt instead of the intended one-shot prompt. It therefore differed from the baseline in two ways at once: it dropped the worked example and it added the target metadata. We added an extra condition to hold the exemplar constant while varying only the target specification and updated the pre-registration with a timestamp, leaving the original hypothesis untouched. The uncorrected prompt is only mentioned in the paper to justify the additional condition.

The second deviation changed the instrument after seeing the results, and it changed the instrument in a way that favoured our method: the Java green-test gate (described above) was updated to comply with the T2 definition. Eight functions were affected under the baseline prompt and ten under the enriched prompt, against two for each of the two Java baselines. Therefore, it improved our results more than those of our competitors. Three safeguards applied: first, the timestamps of the uncorrected version committed to the repository preceded the corrected measurement runs. Second, both settings were reported alongside each other for all Java comparisons. Finally, we only stated conclusions that applied to both; where they did not, we reported the comparison as inconclusive. The Python measurements were not repeated, as they complied with the pre-registered definition from the start.

7 Conclusion

We set out to explain why a language model asked to generate tests for individual functions often fails at doing so, finding that an appreciable part of the cause lies outside the model. Of the 1,848 functions in the medium complexity category across ten projects, only 26 % are simultaneously public and unambiguously named. That is, the rest cannot be legitimately targeted by an external test suite, and a model evaluated upon its ability to find faults in these is penalized for shortcomings unrelated to its own. Recreating the sample under a fixed set of constraints, and evaluating each suite at four nested levels, drastically alters the picture.

Our main finding is a discrepancy that would be masked by a summary using one metric. Supplying the model with the target's fully qualified name, signature, and declaring class, along with a verified constructor call, raises the fraction of Python suites that reach the target from 29/60 to 42/60 ($p = 0.003$) but the fraction that finds an injected fault only from 16/60 to 22/60 ($p = 0.33$). The intervention improves arrival, not discrimination, and would have shown as an overwhelming success if evaluated on the coverage metric alone.

The asymmetry is what allows the same change to benefit Python and not Java. The target block provides identity, and Python uses dynamic binding: fourteen of seventeen import failures vanish. Java binds identity and type at

compile time, and the count of compile failures remains identical (38 before, 38 after, 35 of them the same functions) with no obvious intervention. What remains in both languages is the opportunity to create a receiver in a particular state that reveals behavior, and neither prompt contains such a condition even after the constructor call has been demonstrated.

The comparison with existing generators is not strictly favorable to one language or the other. On Python, our model vastly outperforms Pynguin under both of that suite's configurations ($p \leq 0.003$ in all comparisons); while on Java, both EvoSuite and Randoop detect faults on more targets than our model under either configuration, with only EvoSuite showing a significant improvement, and only under the stricter gate ($p = 0.028$). A study limited to one language would have confidently concluded the opposite of what it found.

At last, and perhaps most unexpectedly, the study that produced these results may be most useful in its ability to reproduce itself. We identified six ways in which a measurement process can fail unnoticed, and we offer this paper as a caution to future work. A hard-coded validity constant, a flag miscounting executions as test cases, reflection turning method names into strings the compiler never checks, two tool-version mismatches that discovered no tests while reporting success, and a green-test gate implemented differently in the two languages account for the six. We introduced four of these defects and fixed the other two when we noticed them; had we trusted one of them to behave as documented, we would have reported a statistically significant improvement in the exact opposite direction. We describe the full funnel and configurations in greater detail for both runs of every tool we tested more than once, and for every changed setting in the gate we repaired, reporting no discovery that did not appear in both.

Future Work

Three issues deserve further immediate attention. First, the state barrier described in Section 5.1, which neither better prompting nor a verified constructor call appear to address, may be eliminated by providing an executed state: a receiver already placed in a state that allows access to the target branches. Second, the Java compile errors should be categorized according to the type-system violations that cause them (Section 5.1), clarifying whether the typing barrier is truly distinct from the addressing barrier or merely a special case. Finally, the funnel analysis (Section 3.1) should be repeated across more projects to confirm whether roughly three quarters of medium-complexity functions are unaddressable from outside the declaring class in more than one language and in more than one paradigm.

References

1. Arcuri, A., Briand, L.: A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. Software Testing, Verification and Reliability **24**(3) (2014)

2. Chen, M., Tworek, J., Jun, H., Yuan, Q., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
3. Fraser, G., Arcuri, A.: EvoSuite: Automatic test suite generation for object-oriented software. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE) (2011)
4. Fraser, G., Arcuri, A.: A large-scale evaluation of automated unit test generation using EvoSuite. ACM Transactions on Software Engineering and Methodology (TOSEM) **24**(2) (2014)
5. Inozemtseva, L., Holmes, R.: Coverage is not strongly correlated with test suite effectiveness. In: Proceedings of the 36th International Conference on Software Engineering (ICSE) (2014)
6. Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G.: Are mutants a valid substitute for real faults in software testing? In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE) (2014)
7. Kerr, N.L.: HARKing: Hypothesizing after the results are known. Personality and Social Psychology Review **2**(3) (1998)
8. Lukasczyk, S., Fraser, G.: Pynguin: Automated unit test generation for Python. In: Proceedings of the 44th International Conference on Software Engineering: Companion Proceedings (ICSE Companion) (2022)
9. Lukasczyk, S., Kroiß, F., Fraser, G.: An empirical study of automated unit test generation for Python. Empirical Software Engineering **28**(2) (2023)
10. McCabe, T.J.: A complexity measure. IEEE Transactions on Software Engineering **SE-2**(4) (1976)
11. Pacheco, C., Lahiri, S.K., Ernst, M.D., Ball, T.: Feedback-directed random test generation. In: Proceedings of the 29th International Conference on Software Engineering (ICSE) (2007)
12. Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., Harman, M.: Mutation testing advances: An analysis and survey. Advances in Computers **112** (2019)
13. Schäfer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering **50**(1) (2024)
14. Shamshiri, S., Just, R., Rojas, J.M., Fraser, G., McMinn, P., Arcuri, A.: Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges. In: Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2015)
15. Siddiq, M.L., Santos, J.C.S., Tanvir, R.H., Ulfat, N., Al Rifat, F., Carvalho Lopes, V.: Using large language models to generate JUnit tests: An empirical study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE) (2024)
16. Tufano, M., Drain, D., Svyatkovskiy, A., Deng, S.K., Sundaresan, N.: Unit test case generation with transformers and focal context. arXiv preprint arXiv:2009.05617 (2020)
17. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, landscape, and vision. IEEE Transactions on Software Engineering **50**(4) (2024)
18. Yuan, Z., Lou, Y., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X.: No more manual tests? evaluating and improving ChatGPT for unit test generation. arXiv preprint arXiv:2305.04207 (2023)